

Lab 1 — NTFS File Server

Active Directory · NTFS · SMB · Terraform + PowerShell through the Azure VM agent

Depends on: Nothing — standalone. Builds every resource it uses.

What gets created: DC01, FS01, CLIENT01, a VNet, an NSG and a Key Vault in RG-FileServerLab, plus a one-time remote state storage account in rg-tfstate

Hands-on time: About 1.5–2 hours. Most of it is waiting on VM provisioning and the DC01 promotion reboot.

Cost: Three Windows VMs bill by the hour while running. Deallocate them whenever you stop (Step 9).

Verified-build edition, September 2026. Rewritten from the actual rebuild: every file below is the version that was deployed, every command is the one that was run, and every error in the troubleshooting section really happened. Where this differs from the original SOP, see “What Changed From the Original SOP” at the end.

Rule #1. Never apply a plan in this project that wants to destroy or replace a VM. It would wipe the whole `lab.local` domain. The one thing that causes it — a missing or wrong admin password variable — is covered in Operating Rules, and it's worth reading before you run anything.

How This Lab Fits Into the Series

Lab 1 builds everything from scratch. Lab 2 layers Azure access control on top of Lab 1's FS01. The Azure Update Manager lab stands alone.

Lab	What it deploys	Relationship
Lab 1 — NTFS File Server	DC01, FS01, CLIENT01, VNet, NSG, Key Vault in RG-FileServerLab	Standalone — creates all of its infrastructure
Lab 2 — Azure RBAC	Three service principals and three role assignments scoped to FS01 — no VMs	Depends on Lab 1 — reads FS01 through Terraform data sources, reuses Lab 1's state storage and Key Vault
AUM — Azure Update Manager	DC01, WS01, WS02, VNet, Key Vault in rg-aumLab	Standalone — independent of Labs 1 and 2

What This Lab Covers and Why It Matters

Every organization has to answer the same question about its shared files: who can read what, and who can change it. Finance data should be visible to Finance. HR files should be invisible to everyone else. The standard answer — a Windows file server, Active Directory security groups, and NTFS permissions granted to those groups — is still what most organizations run, and it's one of the most common interview topics for a systems administrator role.

This lab builds that pattern end to end, then proves it by signing in as four different people and confirming each one gets exactly the access their job needs. No more, no less.

What You Will Learn

Skill	Why it matters in a real environment
Build infrastructure as code with	Terraform describes the whole lab in files you can review, rebuild and destroy. Remote

Skill	Why it matters in a real environment
remote state	state in Azure Storage means the record of what exists isn't tied to one laptop — which is why this lab survived a move to a new machine with zero changes.
Promote a domain controller and build AD from PowerShell	Creating the forest, OUs, groups and users by script is repeatable and reviewable. Clicking through Server Manager isn't.
Grant NTFS permissions to groups, never to users	When someone changes jobs you move them between groups. You never have to re-edit folder permissions.
Understand SMB share vs NTFS permissions	When both apply, the most restrictive wins. Opening the share wide and locking down NTFS gives you one place to manage access.
Keep secrets out of files and command lines	The only password in the lab lives in Key Vault and is read at run time. It's never typed into a script or committed.
Configure VMs through the Azure VM agent	<code>az vm run-command</code> reaches every VM through Azure's control plane, so the NSG can stay locked down to a single RDP rule.
Verify results instead of trusting status messages	"Provisioning succeeded" only means a script launched. This lab checks the actual result after every step — the habit that caught the real failures in this build.

The Business Scenario Behind the Test Users

The four test users represent two departments with different needs. Brian is the deliberate edge case: he belongs to two Finance groups at once, which shows how permissions from multiple groups combine.

User	Department / role	Group membership	Finance	HR
<code>alice.finance</code>	Finance analyst	<code>GG-Finance-ReadOnly</code>	Read only	No access
<code>brian.finance</code>	Finance manager	<code>GG-Finance-ReadOnly</code> and <code>GG-Finance-Modify</code>	Read, edit, create, delete	No access
<code>carla.hr</code>	HR coordinator	<code>GG-HR-ReadOnly</code>	No access	Read only
<code>david.hr</code>	HR manager	<code>GG-HR-FullControl</code>	No access	Full control, including permissions

Why Brian gets Modify. Brian's two memberships don't conflict. When there's no Deny involved, NTFS Allow permissions from every group a user belongs to add together, so his effective access is the union: read plus modify. "Most restrictive wins" is a different rule. It applies between the share and NTFS layers, not between groups.

Architecture

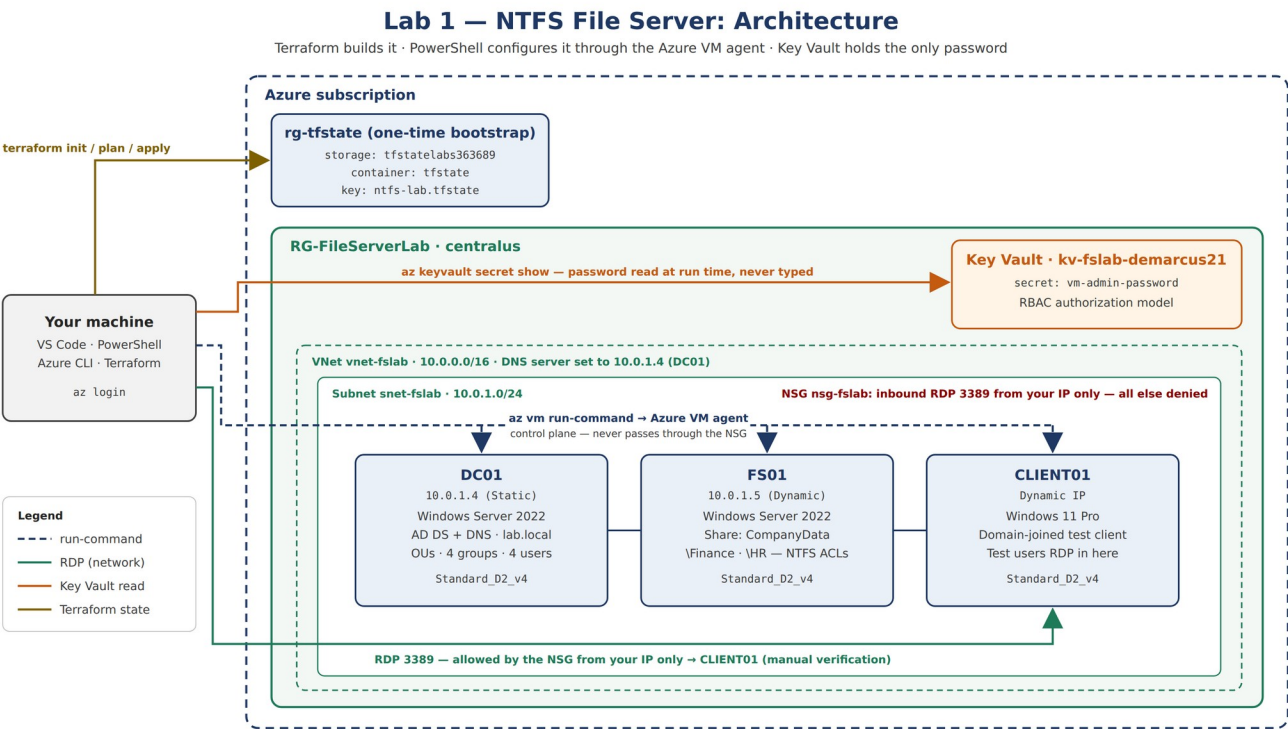


Figure 1 — The NSG admits only RDP from your IP. Configuration reaches the VMs through the Azure VM agent instead, which never touches the network path.

DC01 is the domain controller and DNS server for `lab.local`, with a static private IP so the address every other machine uses for DNS never changes. FS01 hosts the `CompanyData` share with `Finance` and `HR` subfolders. CLIENT01 is the Windows 11 workstation the test users sign in to. All three sit on one subnet behind an NSG with a single inbound rule: RDP from your public IP.

Why Each Component Exists

Component	Why it's needed
rg-tfstate storage account	Holds Terraform's state file remotely. Lab 2 reuses the same container with its own state key, so the two labs don't touch each other's records.
RG-FileServerLab	One container for everything in the lab. Lab 2 finds FS01 by looking inside this group by name, so the name must match exactly.
VNet with custom DNS (dns_servers)	Points every VM at DC01 (10.0.1.4) for DNS. Without it, FS01 and CLIENT01 use Azure's DNS, can't find lab.local, and the domain join fails.
Static private IP on DC01	Every machine's DNS setting points at this address. If it moved after a reboot, name resolution for the whole domain would break.
depends_on on the FS01 and CLIENT01 NICs	Makes the dynamic NICs wait for DC01's static NIC. Without it, Terraform builds all three in parallel and a dynamic NIC can grab 10.0.1.4 first.
NSG — one inbound rule	RDP 3389 from your IP only. Everything else is blocked by Azure's default deny.
Key Vault (RBAC model)	Stores the VM admin password. You read it with <code>az keyvault secret show</code> when

Component	Why it's needed
	you need it, so it never lands in a file or your shell history. Lab 2 stores its service principal secrets here too.
<code>time_sleep</code> before writing the secret	A new role assignment takes a few seconds to take effect. The pause stops the secret write from failing intermittently with an authorization error.
<code>vm_agent_platform_updates_enabled = true</code>	Matches what Azure sets server-side. Without it, the azurearm 3.x provider reports permanent drift on all three VMs (see Troubleshooting).
<code>az vm run-command</code>	Runs PowerShell inside a VM through the Azure VM agent, over Azure's control plane. This is how the lab configures machines while the NSG stays locked down.
OUs, then groups, then users	OUs organize objects but can't be granted permissions. Security groups are what NTFS ACLs point at. Users get access only by being in a group.
Share wide open, NTFS locked down	Share: Everyone Full Control. NTFS: explicit grants per group. Since the most restrictive layer wins, NTFS is the single place access is actually decided.

Prerequisites — Verify Before Starting

Built and verified with Terraform 1.16.2 and Azure CLI 2.90.0 on Windows. On a fresh Windows machine, set up once:

```

winget install --exact --id Microsoft.AzureCLI
winget install --exact --id Hashicorp.Terraform
# Close the terminal and open a new one - an open terminal won't see newly installed programs

az version
terraform -version          # must be >= 1.5.0

# Recent Azure CLI versions use the Windows sign-in broker, which can hang inside VS Code.
# Switch to browser sign-in (one time, persists):
az config set core.enable_broker_on_windows=false

# Windows client editions block all .ps1 files by default:
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned

```

Then sign in and confirm you're on the right subscription:

```

az login
az account show --query "{subscription:name,id:id}" -o table
# Wrong one? az account set --subscription "<name or id>"

# Your public IP, for the NSG rule:
Invoke-RestMethod https://api.ipify.org

# Confirm the Windows 11 image SKU still exists - retired SKUs fail without warning:
az vm image list-skus -l centralus -p MicrosoftWindowsDesktop -f windows-11 -o table

```

Step 1 — Create the Project Folder

```
New-Item -ItemType Directory -Path "$HOME\Projects\ntfs-lab-terraform"
cd "$HOME\Projects\ntfs-lab-terraform"
New-Item -ItemType Directory -Path scripts
```

Finished layout:

```
ntfs-lab-terraform/
├── versions.tf           ← Terraform + provider versions
├── backend.tf           ← remote state location
├── variables.tf         ← inputs; admin_password has no default
├── main.tf              ← network + three VMs
├── keyvault.tf          ← Key Vault + the admin password secret
├── outputs.tf
├── terraform.tfvars.example ← safe template, commit this
├── terraform.tfvars      ← your real values, never commit
├── .gitignore
└── scripts/
    ├── 00-promote-dc.ps1      ← DC01
    ├── 01-join-domain.ps1     ← FS01 and CLIENT01
    ├── 02-create-ous.ps1      ← DC01
    ├── 03-create-groups.ps1   ← DC01
    ├── 04-create-users.ps1    ← DC01
    ├── 05-create-share-and-permissions.ps1 ← FS01
    └── 06-add-rdp-users.ps1    ← CLIENT01
```

Step 2 — One-Time Remote State Setup

Terraform can't create the storage account that holds its own state. That's a chicken-and-egg problem, so you create it once by hand. Lab 2 reuses it.

```
az group create --name rg-tfstate --location centralus
az storage account create --name tfstatelabs<random-digits> --resource-group rg-tfstate --sku
Standard_LRS
az storage container create --name tfstate --account-name tfstatelabs<random-digits>

# Confirm the exact name before putting it in backend.tf:
az storage account list -g rg-tfstate --query "[].name" -o tsv
```

Storage account names are global across all of Azure: 3-24 lowercase letters and digits. This build uses **tfstatelabs363689**. Swap in your own name everywhere it appears.

Step 3 — Create the Terraform Files

File: `versions.tf`

Two different operators on purpose. `required_version` uses `>=` because the Terraform CLI stays backward compatible across 1.x. Providers use `~>` (pessimistic), because a new major provider version can break your config on the next `init`.

```
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    azurearm = {
      source  = "hashicorp/azurearm"
      version = "~> 3.100"
    }
    random = {
      source  = "hashicorp/random"
      version = "~> 3.6"
    }
    time = {
      source  = "hashicorp/time"
      version = "~> 0.11"
    }
  }
}
```

File: `backend.tf`

```
# Bootstrap this manually before `terraform init` - Terraform can't create
# the storage account its own state depends on (see Step 2).
#
# az group create --name rg-tfstate --location centralus
# az storage account create --name <globally-unique-name> --resource-group rg-tfstate --sku
Standard_LRS
# az storage container create --name tfstate --account-name <that-name>
#
# Confirm the real storage account name with `az storage account list`
# before pasting it below.

terraform {
  backend "azurearm" {
    resource_group_name = "rg-tfstate"
    storage_account_name = "tfstatelabs363689"
    container_name       = "tfstate"
    key                  = "ntfs-lab.tfstate"
  }
}
```

File: **variables.tf**

admin_password, **allowed_rdp_source_ip** and **key_vault_name** have no defaults. Terraform stops and asks rather than guessing.

```
variable "resource_group_name" {
  description = "Resource group that holds every resource in this lab"
  type        = string
  default     = "RG-FileServerLab"
}

variable "location" {
  description = "Azure region to deploy into"
  type        = string
  default     = "centralus"
}

variable "admin_username" {
  description = "Local admin username for every VM in the lab"
  type        = string
  default     = "labadmin"
}

variable "admin_password" {
  description = "Local admin password for every VM. Set via $env:TF_VAR_admin_password – never typed into a file."
  type        = string
  sensitive   = true
}

variable "domain_name" {
  description = "Active Directory domain promoted on DC01"
  type        = string
  default     = "lab.local"
}

variable "vnet_address_space" {
  type        = list(string)
  default     = ["10.0.0.0/16"]
}

variable "subnet_address_prefix" {
  type        = list(string)
  default     = ["10.0.1.0/24"]
}

variable "dc01_private_ip" {
  description = "Static private IP reserved for DC01, so DNS never breaks after a reboot"
  type        = string
  default     = "10.0.1.4"
}

variable "allowed_rdp_source_ip" {
  description = "The one public IP the NSG allows through on port 3389 – no default, must be supplied"
  type        = string
}
```

```

variable "server_vm_size" {
  description = "DC01 / FS01 size. Must support Gen2 boot"
  type        = string
  default     = "Standard_D2_v4"
}

variable "client_vm_size" {
  type        = string
  default     = "Standard_D2_v4"
}

variable "client_image_sku" {
  description = "Windows 11 Pro Marketplace SKU – verify with az vm image list-skus first, retired SKUs 404 without warning"
  type        = string
  default     = "win11-24h2-pro"
}

variable "key_vault_name" {
  description = "Must be globally unique across all of Azure, not just this subscription"
  type        = string
}

```

File: **main.tf**

Things worth noticing: **dns_servers** on the VNet, the static IP on DC01's NIC, **depends_on** on the two dynamic NICs, and **vm_agent_platform_updates_enabled = true** on each VM.

```

provider "azurerm" {
  features {}
}

resource "azurerm_resource_group" "lab" {
  name     = var.resource_group_name
  location = var.location
}

resource "azurerm_virtual_network" "lab" {
  name                = "vnet-fslab"
  address_space       = var.vnet_address_space
  location             = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
  dns_servers         = [var.dc01_private_ip]
}

resource "azurerm_subnet" "lab" {
  name                 = "snet-fslab"
  resource_group_name = azurerm_resource_group.lab.name
  virtual_network_name = azurerm_virtual_network.lab.name
  address_prefixes     = var.subnet_address_prefix
}

resource "azurerm_network_security_group" "lab" {
  name                = "nsg-fslab"
  location             = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
}

```



```

security_rule {
  name          = "Allow-RDP-From-Me"
  priority      = 100
  direction     = "Inbound"
  access        = "Allow"
  protocol      = "Tcp"
  source_port_range = "*"
  destination_port_range = "3389"
  source_address_prefix = var.allowed_rdp_source_ip
  destination_address_prefix = "*"
}
}

resource "azurerm_subnet_network_security_group_association" "lab" {
  subnet_id          = azurerm_subnet.lab.id
  network_security_group_id = azurerm_network_security_group.lab.id
}

resource "azurerm_public_ip" "dc01" {
  name          = "pip-dc01"
  location      = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
  allocation_method = "Static"
  sku           = "Standard"
}

resource "azurerm_network_interface" "dc01" {
  name          = "dc01-nic"
  location      = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name

  ip_configuration {
    name          = "internal"
    subnet_id     = azurerm_subnet.lab.id
    private_ip_address_allocation = "Static"
    private_ip_address      = var.dc01_private_ip
    public_ip_address_id    = azurerm_public_ip.dc01.id
  }
}

resource "azurerm_windows_virtual_machine" "dc01" {
  name          = "DC01"
  resource_group_name = azurerm_resource_group.lab.name
  location      = azurerm_resource_group.lab.location
  size          = var.server_vm_size
  admin_username = var.admin_username
  admin_password = var.admin_password
  network_interface_ids = [azurerm_network_interface.dc01.id]
  vm_agent_platform_updates_enabled = true

  os_disk {
    caching          = "ReadWrite"
    storage_account_type = "Standard_LRS"
  }

  source_image_reference {

```

```

    publisher = "MicrosoftWindowsServer"
    offer     = "WindowsServer"
    sku       = "2022-datacenter-azure-edition"
    version   = "latest"
  }
}

resource "azurerm_public_ip" "fs01" {
  name            = "pip-fs01"
  location         = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
  allocation_method = "Static"
  sku              = "Standard"
}

resource "azurerm_network_interface" "fs01" {
  name            = "fs01-nic"
  location         = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name

  ip_configuration {
    name            = "internal"
    subnet_id       = azurerm_subnet.lab.id
    private_ip_address_allocation = "Dynamic"
    public_ip_address_id = azurerm_public_ip.fs01.id
  }

  depends_on = [azurerm_network_interface.dc01]
}

resource "azurerm_windows_virtual_machine" "fs01" {
  name            = "FS01"
  resource_group_name = azurerm_resource_group.lab.name
  location         = azurerm_resource_group.lab.location
  size            = var.server_vm_size
  admin_username   = var.admin_username
  admin_password    = var.admin_password
  network_interface_ids = [azurerm_network_interface.fs01.id]
  vm_agent_platform_updates_enabled = true

  os_disk {
    caching          = "ReadWrite"
    storage_account_type = "Standard_LRS"
  }

  source_image_reference {
    publisher = "MicrosoftWindowsServer"
    offer     = "WindowsServer"
    sku       = "2022-datacenter-azure-edition"
    version   = "latest"
  }
}

resource "azurerm_public_ip" "client01" {
  name            = "pip-client01"
  location         = azurerm_resource_group.lab.location

```

```

    resource_group_name = azurerm_resource_group.lab.name
    allocation_method    = "Static"
    sku                  = "Standard"
}

resource "azurerm_network_interface" "client01" {
  name                = "client01-nic"
  location            = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name

  ip_configuration {
    name                = "internal"
    subnet_id          = azurerm_subnet.lab.id
    private_ip_address_allocation = "Dynamic"
    public_ip_address_id = azurerm_public_ip.client01.id
  }
  depends_on = [azurerm_network_interface.dc01]
}

resource "azurerm_windows_virtual_machine" "client01" {
  name                = "CLIENT01"
  resource_group_name = azurerm_resource_group.lab.name
  location            = azurerm_resource_group.lab.location
  size               = var.client_vm_size
  admin_username     = var.admin_username
  admin_password     = var.admin_password
  network_interface_ids = [azurerm_network_interface.client01.id]
  vm_agent_platform_updates_enabled = true

  os_disk {
    caching          = "ReadWrite"
    storage_account_type = "Standard_LRS"
  }

  source_image_reference {
    publisher = "MicrosoftWindowsDesktop"
    offer     = "windows-11"
    sku       = var.client_image_sku
    version   = "latest"
  }
}

```

File: keyvault.tf

```

data "azurerm_client_config" "current" {}

resource "azurerm_key_vault" "lab" {
  name                = var.key_vault_name
  location            = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
  tenant_id          = data.azurerm_client_config.current.tenant_id
  sku_name            = "standard"

  enable_rbac_authorization = true
}

```

```

}

resource "azurerm_role_assignment" "deployer_secrets_officer" {
  scope           = azurerm_key_vault.lab.id
  role_definition_name = "Key Vault Secrets Officer"
  principal_id    = data.azurerm_client_config.current.object_id
}

# A fresh role assignment can take a few seconds to propagate. Without
# this, the next resource (writing the secret) can fail intermittently
# with an authorization error even though the assignment "exists."
resource "time_sleep" "rbac_propagation" {
  depends_on = [azurerm_role_assignment.deployer_secrets_officer]
  create_duration = "30s"
}

resource "azurerm_key_vault_secret" "vm_admin_password" {
  name       = "vm-admin-password"
  value      = var.admin_password
  key_vault_id = azurerm_key_vault.lab.id

  depends_on = [time_sleep.rbac_propagation]
}

```

File: **outputs.tf**

```

output "resource_group_name" {
  value = azurerm_resource_group.lab.name
}
output "key_vault_name" {
  value = azurerm_key_vault.lab.name
}
output "dc01_public_ip" {
  value = azurerm_public_ip.dc01.ip_address
}
output "dc01_private_ip" {
  value = azurerm_network_interface.dc01.private_ip_address
}
output "fs01_public_ip" {
  value = azurerm_public_ip.fs01.ip_address
}
output "client01_public_ip" {
  value = azurerm_public_ip.client01.ip_address
}

```

File: **terraform.tfvars.example**

```

# Copy to terraform.tfvars (gitignored) and fill in real values.

resource_group_name = "RG-FileServerLab"
location            = "centralus"
admin_username      = "labadmin"
allowed_rdp_source_ip = "203.0.113.10" # your real public IP
key_vault_name       = "kv-fslab-<pick-something-unique>" # globally unique

```

```
# admin_password is deliberately NOT set here.
# Set it per terminal session instead, so it's never written to disk:
# PowerShell: $env:TF_VAR_admin_password = "..."
```

File: .gitignore

Keeps real values and state out of git. The lock file is deliberately committed, because it pins the exact provider versions so a clone rebuilds the same thing.

```
# Real values and state - never commit these
terraform.tfvars
*.tfvars
!terraform.tfvars.example
*.tfstate
*.tfstate.*
.terraform/
*.tfplan
crash.log
crash.*.log

# Commit .terraform.lock.hcl - it pins exact provider versions so a
# clone of this repo reproduces the same build. Don't add it here.
```

Step 4 — Configure Variables

```
Copy-Item terraform.tfvars.example terraform.tfvars
# Edit terraform.tfvars: your public IP, and a globally unique Key Vault name
```

The admin password goes in an environment variable, never a file. It becomes the VM admin password, and in this lab it's also reused for the DSRM (Directory Services Restore Mode) password and the four test users. That's fine for a throwaway lab, and never acceptable in production.

First deployment only — choose the password

```
$env:TF_VAR_admin_password = "<12+ chars, 3 of: upper, lower, digit, symbol>"
```

Every session after that — read it back from Key Vault

```
$env:TF_VAR_admin_password = az keyvault secret show --vault-name kv-fslab-demarcus21 --name vm-admin-
password --query value -o tsv
```

Environment variables disappear when the terminal closes. Every new terminal needs the Key Vault line above before any **plan** or **apply** in this folder. See Operating Rules for what happens if you skip it.

Step 5 — Deploy the Infrastructure

```
terraform init
terraform plan      # expect: Plan: 18 to add, 0 to change, 0 to destroy
terraform apply
```

The 18 resources: resource group, VNet, subnet, NSG, NSG association, three public IPs, three NICs, three VMs, Key Vault, the Key Vault role assignment, the `time_sleep`, and the secret.

- Expect 15–40 minutes. CLIENT01 (Windows 11) is consistently the slowest.
- If CLIENT01 fails with `OSProvisioningTimedOut`, the Windows 11 guest agent didn't report in before Azure's timeout. Run `terraform apply` again. It succeeded on the retry in this build.
- If a NIC fails with `PrivateIPAddressIsAllocated`, see Troubleshooting. The `depends_on` in `main.tf` exists to prevent it.

Confirm what actually got built, rather than trusting the apply message:

```
terraform output
az vm list -d -g RG-FileServerLab --query "[].{name:name,status:powerState,ip:privateIps}" -o table
```

`az vm list` only returns `powerState` with `-d (--show-details)`. Without it the column comes back empty, and that looks like a problem when it isn't.

Step 6 — Build the Domain and the File Server

Seven scripts, run one at a time through `az vm run-command`, each followed by a check of the actual result. Run everything from inside the `scripts` folder, and load the lab password once per terminal:

```
cd "$HOME\Projects\ntfs-lab-terraform\scripts"
$labPwd = az keyvault secret show --vault-name kv-fslab-demarcus21 --name vm-admin-password --query value -o tsv
```

Gotcha 1. If the file after `@` in `--scripts "@file.ps1"` isn't found in the current folder, Azure CLI doesn't report a missing file. It silently sends the literal text `@file.ps1` to the VM as the script, and PowerShell fails with a confusing “splating operator” error. In this build the cause was a one-letter typo in a filename. When you see that error, run `dir` before anything else.

Gotcha 2. A run-command result that says “Provisioning succeeded” only means the VM agent launched the script. It says nothing about whether the script worked. In this build DC01 “succeeded” without ever being promoted, and it surfaced two steps later as a failed domain join. Every step below ends with a verification command. Don't skip them.

6.0 — Promote DC01 to domain controller

Installs AD DS (Active Directory Domain Services) and creates the new `lab.local` forest with DNS. Target: **DC01**.

File: `scripts/00-promote-dc.ps1`

```
<#
00-promote-dc.ps1
```

```

Promotes DC01 into a brand-new Active Directory forest, installing
DNS as part of the same operation. This server has no domain to
join yet – it's creating the very first one.
#>

param(
    [Parameter(Mandatory = $true)]
    [string]$SafeModePassword,

    [string]$DomainName = "lab.local",
    [string]$DomainNetbiosName = "LAB"
)

$secureSafeModePassword = ConvertTo-SecureString $SafeModePassword -AsPlainText -Force

# Step 1: install the AD DS (Active Directory Domain Services) role itself –
# it doesn't exist on a fresh Windows Server until this runs.
Install-WindowsFeature -Name AD-Domain-Services -IncludeManagementTools

# Step 2: load the promotion cmdlets – only available now that the feature above exists.
Import-Module ADDSDeployment

# Step 3: create the forest, the domain inside it, and install DNS in one operation.
Install-ADDSForest `
    -DomainName $DomainName `
    -DomainNetbiosName $DomainNetbiosName `
    -SafeModeAdministratorPassword $secureSafeModePassword `
    -InstallDns `
    -Force

```

```

az vm run-command invoke -g RG-FileServerLab -n DC01 --command-id RunPowerShellScript --scripts "@00-
promote-dc.ps1" --parameters "SafeModePassword=$labPwd"

```

DC01 reboots automatically at the end of promotion. Wait 5–10 minutes, then verify:

```

az vm run-command invoke -g RG-FileServerLab -n DC01 --command-id RunPowerShellScript --scripts "Get-
WindowsFeature AD-Domain-Services; (Get-CimInstance Win32_ComputerSystem).Domain; (Get-CimInstance
Win32_ComputerSystem).PartOfDomain; Get-Service DNS"

```

Expected: **AD-Domain-Services** Installed · **lab.local** · **True** · DNS service Running. If DNS comes back
 “Cannot find any service with service name 'DNS'”, DC01 was never promoted, whatever the earlier JSON said.

6.1 — Join FS01 and CLIENT01 to the domain

Target: **FS01**, then **CLIENT01**. First confirm each machine can actually find the domain. That means DNS pointing at DC01, and the domain controller locator record answering:

```

az vm run-command invoke -g RG-FileServerLab -n FS01 --command-id RunPowerShellScript --scripts "Get-
DnsClientServerAddress -AddressFamily IPv4 | Format-Table -AutoSize; Resolve-DnsName -Name
_ldap._tcp.dc._msdcs.lab.local -Type SRV"

```

You want DNS server **10.0.1.4** and an SRV record pointing at DC01. A timeout here means DC01 isn't serving the domain yet. Don't attempt the join until it answers.

File: **scripts/01-join-domain.ps1**

```
<#
01-join-domain.ps1
Joins this machine to the lab.local domain. Requires the machine's
DNS to already be pointed at DC01 (see main.tf's dns_servers) –
otherwise it can't find the domain to join in the first place.
#>

param(
    [Parameter(Mandatory = $true)]
    [string]$DomainName,      # e.g. "lab.local"

    [Parameter(Mandatory = $true)]
    [string]$AdminUsername,   # domain admin username, e.g. "labadmin"

    [Parameter(Mandatory = $true)]
    [string]$AdminPassword
)

$securePassword = ConvertTo-SecureString $AdminPassword -AsPlainText -Force
$credential = New-Object System.Management.Automation.PSCredential("$AdminUsername@$DomainName",
$securePassword)

Add-Computer -DomainName $DomainName -Credential $credential -Restart -Force
```

```
az vm run-command invoke -g RG-FileServerLab -n FS01 --command-id RunPowerShellScript --scripts "@01-join-domain.ps1" --parameters "DomainName=lab.local" "AdminUsername=labadmin" "AdminPassword=$labPwd"
az vm run-command invoke -g RG-FileServerLab -n CLIENT01 --command-id RunPowerShellScript --scripts "@01-join-domain.ps1" --parameters "DomainName=lab.local" "AdminUsername=labadmin" "AdminPassword=$labPwd"
```

Both machines restart. Give them a few minutes, then verify each:

```
az vm run-command invoke -g RG-FileServerLab -n FS01 --command-id RunPowerShellScript --scripts "(Get-CimInstance Win32_ComputerSystem).Domain; (Get-CimInstance Win32_ComputerSystem).PartOfDomain"
az vm run-command invoke -g RG-FileServerLab -n CLIENT01 --command-id RunPowerShellScript --scripts "(Get-CimInstance Win32_ComputerSystem).Domain; (Get-CimInstance Win32_ComputerSystem).PartOfDomain"
```

Expected for both: **lab.local** and **True**.

6.2 — Create the OU structure

Target: DC01.

File: **scripts/02-create-ous.ps1**

```
<#
02-create-ous.ps1
Builds the OU (Organizational Unit) structure that will hold this
```


lab's security groups and test user accounts, separate from AD's default containers. Run on DC01 – the only machine with the ActiveDirectory PowerShell module available by default.

```
#>
```

```
Import-Module ActiveDirectory
```

```
$domainDN = (Get-ADDomain).DistinguishedName # e.g. "DC=lab,DC=local"
```

```
New-ADOrganizationalUnit -Name "FileServerLab" -Path $domainDN
```

```
New-ADOrganizationalUnit -Name "Groups" -Path "OU=FileServerLab,$domainDN"
```

```
New-ADOrganizationalUnit -Name "Users" -Path "OU=FileServerLab,$domainDN"
```

```
az vm run-command invoke -g RG-FileServerLab -n DC01 --command-id RunPowerShellScript --scripts "@02-create-ous.ps1"
```

```
az vm run-command invoke -g RG-FileServerLab -n DC01 --command-id RunPowerShellScript --scripts "Get-ADOrganizationalUnit -Filter * | Select-Object Name, DistinguishedName"
```

Expected: **FileServerLab**, **Groups** and **Users**, the last two under **OU=FileServerLab,DC=lab,DC=local**.

6.3 — Create the security groups

Target: **DC01**.

File: **scripts/03-create-groups.ps1**

```
<#
```

```
03-create-groups.ps1
```

```
Creates the security groups this lab uses to control NTFS folder access. These live in the Groups OU built by 02-create-ous.ps1 – the NTFS ACL created by 05-create-share-and-permissions.ps1 grants permissions to these groups, never directly to individual users.
```

```
#>
```

```
Import-Module ActiveDirectory
```

```
$groupOU = "OU=Groups,OU=FileServerLab,DC=lab,DC=local"
```

```
$groups = @(
    "GG-Finance-ReadOnly",
    "GG-Finance-Modify",
    "GG-HR-ReadOnly",
    "GG-HR-FullControl"
)
```

```
foreach ($groupName in $groups) {
    New-ADGroup -Name $groupName -GroupScope Global -GroupCategory Security -Path $groupOU
}
```

```
az vm run-command invoke -g RG-FileServerLab -n DC01 --command-id RunPowerShellScript --scripts "@03-create-groups.ps1"
```

```
az vm run-command invoke -g RG-FileServerLab -n DC01 --command-id RunPowerShellScript --scripts "Get-ADGroup -Filter * -SearchBase 'OU=Groups,OU=FileServerLab,DC=lab,DC=local' | Select-Object Name, GroupScope, GroupCategory"
```

Expected: four groups, each Global / Security.

6.4 — Create the test users

Target: DC01.

File: **scripts/04-create-users.ps1**

```
<#
    04-create-users.ps1
    Creates test user accounts and adds each one to the group(s) that
    determine their effective NTFS permissions later. Brian is
    deliberately placed in two Finance groups at once – the
    cumulative-Allow case: no Deny involved, so his effective access
    becomes the union of both groups' rights.
#>

param(
    [Parameter(Mandatory = $true)]
    [string]$UserPassword
)

Import-Module ActiveDirectory

$usersOU = "OU=Users,OU=FileServerLab,DC=lab,DC=local"
$securePassword = ConvertTo-SecureString $UserPassword -AsPlainText -Force

$users = @(
    @{ Name = "Alice Finance"; SamAccountName = "alice.finance"; Groups = @("GG-Finance-ReadOnly") },
    @{ Name = "Brian Finance"; SamAccountName = "brian.finance"; Groups = @("GG-Finance-ReadOnly",
    "GG-Finance-Modify") },
    @{ Name = "Carla HR"; SamAccountName = "carla.hr"; Groups = @("GG-HR-ReadOnly") },
    @{ Name = "David HR"; SamAccountName = "david.hr"; Groups = @("GG-HR-FullControl") }
)

foreach ($user in $users) {
    New-ADUser `
        -Name $user.Name `
        -SamAccountName $user.SamAccountName `
        -UserPrincipalName "$($user.SamAccountName)@lab.local" `
        -Path $usersOU `
        -AccountPassword $securePassword `
        -Enabled $true `
        -ChangePasswordAtLogon $false

    foreach ($groupName in $user.Groups) {
        Add-ADGroupMember -Identity $groupName -Members $user.SamAccountName
    }
}
```

```
az vm run-command invoke -g RG-FileServerLab -n DC01 --command-id RunPowerShellScript --scripts "@04-
create-users.ps1" --parameters "UserPassword=$labPwd"
az vm run-command invoke -g RG-FileServerLab -n DC01 --command-id RunPowerShellScript --scripts "Get-
ADGroupMember -Identity 'GG-Finance-ReadOnly' | Select-Object Name; Get-ADGroupMember -Identity 'GG-
Finance-Modify' | Select-Object Name; Get-ADGroupMember -Identity 'GG-HR-ReadOnly' | Select-Object
Name; Get-ADGroupMember -Identity 'GG-HR-FullControl' | Select-Object Name"
```

Expected, in order: Alice and Brian · Brian · Carla · David. Brian shows up twice because he's in both Finance groups.

6.5 — Create the share and NTFS permissions

Target: **FS01**. **icacls /inheritance:r** strips every inherited permission, including SYSTEM's and the administrators', which is why the script immediately grants those back. On a production server, forgetting that line can lock everyone out of a folder.

File: **scripts/05-create-share-and-permissions.ps1**

```
<#
    05-create-share-and-permissions.ps1
    Builds the shared folder on FS01 and locks it down using NTFS
    permissions granted to the groups from 03-create-groups.ps1.
    Share (SMB - Server Message Block) permissions are left wide open
    on purpose - since the "most restrictive wins" rule between NTFS
    and Share always applies, opening the Share side all the way
    means NTFS is the only thing actually doing access control.
#>

$rootPath    = "C:\CompanyData"
$financePath = "$rootPath\Finance"
$hrPath      = "$rootPath\HR"

New-Item -Path $rootPath -ItemType Directory -Force | Out-Null
New-Item -Path $financePath -ItemType Directory -Force | Out-Null
New-Item -Path $hrPath -ItemType Directory -Force | Out-Null

New-SmbShare -Name "CompanyData" -Path $rootPath -FullAccess "Everyone"

# Finance: strip inherited permissions, grant back only what's needed
icacls $financePath /inheritance:r
icacls $financePath /grant "SYSTEM:(F)"
icacls $financePath /grant "LAB\Domain Admins:(F)"
icacls $financePath /grant "LAB\GG-Finance-ReadOnly:(RX)"
icacls $financePath /grant "LAB\GG-Finance-Modify:(M)"

# HR: same pattern
icacls $hrPath /inheritance:r
icacls $hrPath /grant "SYSTEM:(F)"
icacls $hrPath /grant "LAB\Domain Admins:(F)"
icacls $hrPath /grant "LAB\GG-HR-ReadOnly:(RX)"
icacls $hrPath /grant "LAB\GG-HR-FullControl:(F)"
```

```
az vm run-command invoke -g RG-FileServerLab -n FS01 --command-id RunPowerShellScript --scripts "@05-
```

```
create-share-and-permissions.ps1"
az vm run-command invoke -g RG-FileServerLab -n FS01 --command-id RunPowerShellScript --scripts
"icacls C:\CompanyData\Finance; icacls C:\CompanyData\HR; Get-SmbShare -Name CompanyData"
```

Expected: exactly four entries per folder, with no leftover inherited ones. Finance: SYSTEM (F), LAB\Domain Admins (F), GG-Finance-ReadOnly (RX), GG-Finance-Modify (M). HR: SYSTEM (F), LAB\Domain Admins (F), GG-HR-ReadOnly (RX), GG-HR-FullControl (F).

6.6 — Allow the test users to RDP into CLIENT01

Target: **CLIENT01**. Being a domain user with rights on a file share doesn't let you sign in to a machine remotely. That's a separate right, controlled by the local Remote Desktop Users group.

File: **scripts/06-add-rdp-users.ps1**

```
<#
06-add-rdp-users.ps1
Grants the test users RDP (Remote Desktop Protocol) login rights
on CLIENT01 by adding them to the local Remote Desktop Users
group. Being a domain user with NTFS permissions on a share
doesn't automatically mean you're allowed to log on to a machine
remotely - that's a completely separate right.
#>

$rdpGroup = "Remote Desktop Users"
$users = @(
    "LAB\alice.finance",
    "LAB\brian.finance",
    "LAB\carla.hr",
    "LAB\david.hr"
)

foreach ($user in $users) {
    Add-LocalGroupMember -Group $rdpGroup -Member $user
}
```

```
az vm run-command invoke -g RG-FileServerLab -n CLIENT01 --command-id RunPowerShellScript --scripts
"@06-add-rdp-users.ps1"
az vm run-command invoke -g RG-FileServerLab -n CLIENT01 --command-id RunPowerShellScript --scripts
"Get-LocalGroupMember -Group 'Remote Desktop Users'"
```

Expected: LAB\alice.finance, LAB\brian.finance, LAB\carla.hr, LAB\david.hr.

Step 7 — Verify as Each User

This is the real test. Everything so far proved the objects exist. This step proves the permissions do what the business needs.

```
terraform output client01_public_ip
mstsc /v:<client01_public_ip>
```

Sign in as each user in turn, `LAB\alice.finance` and so on, using the lab password. Open `\\FS01\CompanyData`, try each folder, and try to create, edit and delete a file. Sign out completely between users.

Sign in as	Folder	Expected	Why
<code>LAB\alice.finance</code>	Finance	Read only — can open, can't save or create	<code>GG-Finance-ReadOnly</code> (RX)
<code>LAB\alice.finance</code>	HR	Access denied	In neither HR group
<code>LAB\brian.finance</code>	Finance	Read, edit, create, delete	Allows from both Finance groups combine
<code>LAB\carla.hr</code>	HR	Read only	<code>GG-HR-ReadOnly</code> (RX)
<code>LAB\carla.hr</code>	Finance	Access denied	In neither Finance group
<code>LAB\david.hr</code>	HR	Full control, including changing permissions	<code>GG-HR-FullControl</code> (F)

All results matched the table in this build. If RDP itself times out, check that CLIENT01 is running and that your public IP still matches `allowed_rdp_source_ip`.

Step 8 — Operating Rules for This Project

These rules come straight from mistakes made or nearly made during this build.

1. Read the Key Vault password into the variable before every plan or apply

`admin_password` is a ForceNew attribute on `azurerm_windows_virtual_machine`. Azure can't change a Windows VM's original admin password through the VM resource, so if the value Terraform sees differs from state, the only way it can make them match is to destroy the VM and build a new one.

That nearly happened here. After moving the project folder, a new terminal had no `TF_VAR_admin_password`, so Terraform prompted for it, and the value typed didn't match. The plan came back as **3 to add, 1 to change, 3 to destroy**, and one line explained why:

```
~ admin_password = (sensitive value) # forces replacement
```

Applying it would have destroyed all three VMs and with them the `lab.local` domain, every OU, group and user, the share, and every NTFS permission. Reading the Key Vault value back into the variable turned the plan into **No changes**.

2. Read the plan, not the summary line

A healthy plan in this folder says **No changes. Your infrastructure matches the configuration**. Any VM marked **must be replaced** or **forces replacement** means stop and fix the cause. Never type yes.

3. Verify every step against the real result

Terraform reporting created, a script reporting succeeded, a command printing a table: none of those is proof. Ask the system directly, from the point of view of the thing that's supposed to work.

4. Moving the project to another folder or another computer

State lives in Azure, so nothing authoritative is on your disk. Delete the `.terraform\` folder before copying (it holds downloaded provider binaries, which `init` rebuilds). Keep `.terraform.lock.hcl`. Don't keep live Terraform folders inside OneDrive, because sync locks files mid-`init`. On the new machine:

```
az login
terraform init
$env:TF_VAR_admin_password = az keyvault secret show --vault-name kv-fslab-demarcus21 --name vm-admin-
password --query value -o tsv
terraform plan      # must say: No changes
```

This exact sequence moved the lab to a new laptop in this build with no changes.

Step 9 — Pause, Resume, or Tear Down

Pause — stop compute billing

```
az vm deallocate --ids $(az vm list -g RG-FileServerLab --query "[].id" -o tsv)
az vm list -d -g RG-FileServerLab --query "[].{name:name,status:powerState}" -o table  # want: VM
deallocated
```

Only deallocate stops compute charges. `az vm stop` powers the OS down but keeps the hardware reserved and billing. Disks and static public IPs still bill a small amount while deallocated.

Resume

```
az vm start --ids $(az vm list -g RG-FileServerLab --query "[].id" -o tsv)
```

Everything comes back exactly as it was: domain, users, share, permissions. Deallocating doesn't touch the disks.

Tear down completely

If Lab 2 is deployed, tear it down first. Its role assignments sit on FS01, and its state would otherwise describe things that no longer exist.

```
$env:TF_VAR_admin_password = az keyvault secret show --vault-name kv-fslab-demarcus21 --name vm-admin-
password --query value -o tsv
terraform destroy
```

Leave `rg-tfstate` in place if you'll run Lab 2 or rebuild later. If a rebuild complains the Key Vault name is taken, the old vault is sitting in soft-delete: check with `az keyvault list-deleted` and remove it with `az keyvault purge --name <vault>`.

Troubleshooting

Every row below happened during this build.

Symptom	Cause	Fix
<code>init/plan</code> errors after editing <code>versions.tf</code>	Operators swapped: <code>>=</code> on providers, or <code>~> 1.5.0</code> on <code>required_version</code> (which blocks newer Terraform CLIs)	<code>required_version = ">= 1.5.0";</code> providers use <code>~></code>
"Duplicate resource ... configuration"	A copy-pasted block (here, the FS01 NIC twice)	Delete the duplicate
<code>PrivateIpAddressIsAllocated</code> on DC01's NIC	A dynamic NIC was created first and took <code>10.0.1.4</code>	<code>depends_on = [azurerm_network_interface.dc01]</code> on the dynamic NICs. If a NIC already in state holds the address, <code>depends_on</code> won't move it: delete that VM, then its NIC, and re-apply
<code>NicInUse</code> when deleting a NIC	It's still attached to a VM	Delete the VM first, then the NIC
<code>OSProvisioningTimedOut</code> on CLIENT01	Windows 11 guest agent slow to report ready	Run <code>terraform apply</code> again
run-command: "splatting operator" parse error	File after <code>@</code> not found in the current folder (typo or wrong directory)	<code>dir</code> in <code>scripts</code> ; fix the name or <code>cd</code> there
run-command says succeeded, nothing changed	The status only means the script launched	Run the verification command for that step
Domain join: "domain does not exist"	DC01 not actually promoted, or the member VM's DNS isn't <code>10.0.1.4</code>	Verify 6.0; check <code>Get-DnsClientServerAddress</code> and the SRV lookup
SRV lookup times out from FS01	DC01 isn't serving <code>lab.local</code> yet	Confirm DNS is Running on DC01; wait for the post-promotion reboot
Plan wants to replace all three VMs	<code>admin_password</code> in the variable differs from state (<code># forces replacement</code>)	Read it from Key Vault into <code>TF_VAR_admin_password</code> ; plan again. Never apply
Permanent drift: <code>vm_agent_platform_updates_enabled</code> true → false	Azure enables it server-side; <code>azurerm 3.x</code> reads it incorrectly (issue #27484, fixed in v4.26.0)	Set <code>vm_agent_platform_updates_enabled = true</code> on each VM (already in <code>main.tf</code>)
<code>az login</code> hangs after choosing an account	Windows sign-in broker stalls inside VS Code	<code>az config set core.enable_broker_on_windows=false</code>
<code>az / terraform</code> not recognized after installing	The terminal was open before the install	Open a new terminal
"Running scripts is disabled on this system"	Execution policy <code>Restricted</code>	<code>Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned</code>
<code>az vm list</code> shows no power state	<code>powerState</code> needs <code>-d</code>	<code>az vm list -d ...</code>

Symptom	Cause	Fix
RDP to CLIENT01 times out	VM deallocated, or your public IP changed	Start the VM; update <code>allowed_rdp_source_ip</code> and apply
Test user can't sign in over RDP	Not in CLIENT01's Remote Desktop Users	Re-run 6.6 and verify

What Changed From the Original SOP

The original SOP was the starting point. The build below diverged from it deliberately.

Area	Original SOP	This build	Why
Test users	Five users: Sarah, Mike, Lisa, John, Tom	Four users: Alice, Brian, Carla, David	Two departments are enough to prove isolation; Brian isolates the cumulative-Allow case
Shares	Four SMB shares	One <code>CompanyData</code> share with <code>Finance</code> and <code>HR</code> subfolders	One share, one wide-open share ACL, NTFS as the single gate
Configuration	One orchestration script (<code>configure-lab.ps1</code>) running everything	Seven scripts run one at a time, each verified	The "Provisioning succeeded" failure showed why every step needs its own check
RDP rights	A Group Policy Object plus a VM extension	Local Remote Desktop Users group via script 06	Simpler, and directly verifiable
VM size	<code>Standard_B2as_v2</code>	<code>Standard_D2_v4</code>	The size this build was deployed and verified on (<code>variables.tf</code> notes it must support Gen2 boot)
Timing	<code>time_sleep</code> waits after the VNet and NSG	<code>depends_on</code> only where a real race existed (the NICs); <code>time_sleep</code> only for Key Vault RBAC	Explicit ordering where it matters, nothing arbitrary
VNet DNS	Not set	<code>dns_servers = [var.dc01_private_ip]</code>	Without it, domain join can't find <code>lab.local</code>
Names	<code>RG-TerraformState</code> , <code>ntfs-lab.terraform.tfstate</code> , <code>Subnet-Servers</code>	<code>rg-tfstate</code> , <code>ntfs-lab.tfstate</code> , <code>snet-fslab</code>	Names as actually deployed
Provider	<code>azurerm ~> 3.0</code>	<code>azurerm ~> 3.100</code> (lock resolves 3.117.1), plus the drift fix	Newer 3.x pin; the drift fix is new

Lessons Worth Carrying Forward

- "It said it worked" and "it works" are different claims. Verify the result, not the status message.
- Read every Terraform plan line by line. One `# forces replacement` line can hide inside a routine-looking change.

- Secrets belong in Key Vault. Because the password was there, a new terminal, a folder move and a new laptop were all recoverable.
- Grant access to groups, and put people in groups. Permissions follow the job, not the person.
- The share and NTFS are two layers. Open one and control access with the other.